Análisis Comparativo de Arquitecturas LSTM y Transformer para la Generación de Melodías Musicales

Deep Learning - Proyecto Final

Alberto Bartolomé Iruela
Universidad de Málaga

Enero 2026

Índice

1. Objetivo / Hipótesis **3**
1.1. Hipótesis Principal . . . 3
1.2. Preguntas de Investigación . . . 3
1.3. Métricas de Evaluación . . . 3

2. Introducción **3**
2.1. Contexto . . . 3
2.2. Motivación . . . 3
2.3. Modelos Seleccionados . . . 4
2.3.1. Preprocesamiento de Datos . . . 4
2.3.2. LSTM (PerformanceRNN) . . . 4
2.3.3. Transformer con RPR (Music Transformer) . . . 4

3. Metodología **5**
3.1. Dataset: MAESTRO v3.0.0 . . . 5
3.1.1. ¿Por qué no utilizamos el conjunto de Test? . . . 5
3.2. Codificación de Eventos . . . 6
3.2.1. Explicación detallada del proceso . . . 6
3.3. Arquitectura LSTM (musiclstm) . . . 7
3.4. Arquitectura Transformer (musictransformer) . . . 7
3.5. Entorno de Entrenamiento . . . 8
3.6. Pipeline Completo: Del Entrenamiento a la Generación . . . 8
3.7. Comandos de Entrenamiento . . . 8
3.7.1. Preprocesamiento del Dataset . . . 8
3.7.2. Entrenamiento LSTM . . . 8
3.7.3. Entrenamiento Transformer . . . 9
3.8. Comandos de Generación . . . 9
3.8.1. Generación con LSTM . . . 9
3.8.2. Generación con Transformer . . . 9

4. Investigación y Análisis **10**
4.1. Resultados del Entrenamiento 10
4.2. Interpretación de Métricas 10
4.2.1. Accuracy 10
4.2.2. Perplexity 10
4.3. Curvas de Entrenamiento 11
4.4. Análisis de Convergencia 12
4.4.1. LSTM 12
4.4.2. Transformer 12
4.5. Eficiencia Computacional 13
4.6. Generación de Muestras 13
4.6.1. Evaluación Cualitativa 13
4.7. Análisis por Longitud de Secuencia 14
4.7.1. Observaciones por Longitud 14
4.7.2. Conclusión del Análisis por Longitud 15

5. Conclusiones **15**
5.1. Hallazgos Principales 15
5.2. Consideraciones sobre la Comparación 16
5.3. Limitaciones del Estudio 16
5.4. Extensiones Futuras 16
5.5. Conclusión Final 17

1. Objetivo / Hipótesis

1.1. Hipótesis Principal

Los modelos **Transformer** superarán a las arquitecturas **LSTM** en la generación de melodías musicales debido a su mecanismo de atención global, que permite capturar dependencias a largo plazo en secuencias musicales de manera más efectiva.

1.2. Preguntas de Investigación

1. ¿Qué arquitectura logra mejor precisión en la predicción del siguiente evento musical?
2. ¿Cómo se compara la perplejidad entre ambos modelos?
3. ¿Cuál es la relación entre el número de parámetros y el rendimiento?
4. ¿Qué modelo genera melodías más coherentes cualitativamente?

1.3. Métricas de Evaluación

- **Cross-Entropy Loss**: Mide el error de predicción (menor es mejor)
- **Accuracy**: Porcentaje de predicciones correctas sobre 388 clases
- **Perplexity**: e^{loss} - representa el número efectivo de opciones entre las que el modelo está indeciso (menor es mejor)

2. Introducción

2.1. Contexto

La generación automática de música es un problema desafiante en el campo del aprendizaje profundo. A diferencia del texto, la música tiene estructura temporal compleja, con dependencias tanto locales (notas consecutivas) como globales (temas o, como se conoce en música clásica, *motivos*, que se repiten a lo largo de una pieza).

2.2. Motivación

Históricamente, las redes recurrentes (RNN/LSTM) han sido el enfoque dominante para modelar secuencias. Sin embargo, los Transformers han revolucionado el procesamiento de lenguaje natural gracias a su mecanismo de atención. Este proyecto investiga si esta innovación se traduce también al dominio musical, evaluando si los Transformers consiguen generar música cualitativamente mejor y más coherente que las arquitecturas recurrentes tradicionales.

2.3. Modelos Seleccionados

Para este estudio se han seleccionado las arquitecturas propuestas por **Google Magenta**, ya que representan el estado del arte en generación musical tanto con LSTM como con Transformers. Aunque las implementaciones originales están desarrolladas en TensorFlow, se ha realizado una **adaptación completa a PyTorch**, un framework más moderno y con mejor rendimiento en GPUs actuales.

Esta adaptación incluye:

- Reimplementación de ambas arquitecturas desde cero en PyTorch
- Adición de métricas de evaluación (accuracy, perplexity) no presentes en las implementaciones originales
- Creación del pipeline de entrenamiento completo con logging y visualización

Los **hiperparámetros** utilizados son los propuestos en los papers originales y en los repositorios oficiales de Magenta [6, 7].

2.3.1. Preprocesamiento de Datos

El preprocesamiento de los archivos MIDI es la parte más crítica del pipeline y se ha tomado como referencia directa de los modelos de Magenta. Este proceso convierte archivos MIDI en secuencias de eventos discretos, lo cual es fundamental para que los modelos puedan aprender patrones musicales. La codificación utiliza 388 clases que representan notas, tiempos y velocidades, permitiendo una representación expresiva de la música.

2.3.2. LSTM (PerformanceRNN)

Basado en la arquitectura de Google Magenta [3], PerformanceRNN utiliza una pila de capas LSTM para modelar secuencias de eventos MIDI. Es el enfoque clásico para generación musical secuencial, capaz de capturar dependencias temporales mediante su estado oculto recurrente.

2.3.3. Transformer con RPR (Music Transformer)

Music Transformer [1] adapta la arquitectura Transformer para música, incorporando **Relative Position Representations (RPR)** [5].

¿Qué es RPR y por qué es útil en música? RPR es una técnica de codificación posicional que representa las **distancias relativas** entre tokens en lugar de sus posiciones absolutas. En el Music Transformer, RPR modifica el cálculo de atención añadiendo información posicional relativa directamente en los scores de atención:

$$S_{rel} = QK^T + QR^T \tag{1}$$

donde $R_{i,j}$ representa embeddings que codifican la distancia relativa entre la posición i del query y la posición j del key.

Las ventajas de RPR sobre la codificación posicional absoluta son:

- **Generalización a secuencias largas**: El modelo aprende patrones basados en distancias ("tres posiciones atrás") en lugar de posiciones específicas, permitiendo generalizar a secuencias más largas que las vistas durante el entrenamiento.

- **Captura de relaciones musicales**: En música, las relaciones temporales y rítmicas entre notas son cruciales independientemente de su posición absoluta en la pieza. Un intervalo de quinta siempre suena igual, esté al principio o al final.
- **Eficiencia**: Se utiliza un número fijo de embeddings únicos donde embeddings idénticos representan la misma distancia relativa en diferentes partes de la secuencia.

3. Metodología

3.1. Dataset: MAESTRO v3.0.0

El dataset MAESTRO (MIDI and Audio Edited for Synchronous TRacks and Organization) [2] contiene:

- **1,276 archivos MIDI** de interpretaciones de piano clásico
- Grabaciones de alta calidad del International Piano-e-Competition
- División predefinida: train/validation/test
- Información de velocidad y timing preciso

3.1.1. ¿Por qué no utilizamos el conjunto de Test?

En tareas de **generación de música**, el conjunto de test tiene un rol fundamentalmente diferente al de tareas de clasificación o regresión. Esta distinción es importante y explica por qué Google Magenta y otros proyectos de generación musical no reportan métricas sobre el conjunto de test.

Diferencia con tareas de clasificación: En clasificación (ej: análisis de sentimiento), el test set permite medir accuracy contra etiquetas "ground truth". La pregunta es: *"¿El modelo predice correctamente la etiqueta?"*.

En **generación musical**, no existe un "ground truth". La pregunta no es *"¿El modelo genera exactamente esta pieza?"* sino *"¿El modelo genera música coherente y de calidad?"*. Evaluar en test solo mediría qué tan bien el modelo predice las notas *específicas* del test set, no su capacidad generativa real.

¿Qué medimos realmente? Las métricas de loss, accuracy y perplexity durante entrenamiento y validación miden la **capacidad del modelo para aprender patrones musicales**:

- **Train**: Qué tan bien el modelo aprende los patrones del dataset.
- **Validation**: Qué tan bien generaliza a secuencias no vistas (detecta overfitting).

Pero la verdadera evaluación de un modelo generativo es la **calidad de las muestras generadas**, que se evalúa:

- **Cualitativamente**: Escuchando las piezas generadas (coherencia, musicalidad).
- **Mediante estudios de usuario**: Evaluaciones por músicos o oyentes.

Práctica en Google Magenta: Los modelos de referencia (PerformanceRNN, Music Transformer) siguen esta misma metodología: entrenan con train, monitorizan con validation, y evalúan la calidad mediante las muestras generadas, no mediante métricas en test.

3.2. Codificación de Eventos

La codificación de eventos convierte archivos MIDI (formato binario con información de notas, tiempos y velocidades) en secuencias de tokens discretos que los modelos pueden procesar. Ambos modelos utilizan la misma codificación de 388 clases, tomada directamente del preprocesador de Magenta:

Rango	Tipo de Evento	Descripción
0-127	NOTE_ON	Inicio de nota (128 pitches MIDI)
128-255	NOTE_OFF	Fin de nota (128 pitches MIDI)
256-355	TIME_SHIFT	Desplazamiento temporal (100×10ms)
356-387	VELOCITY	Velocidad/intensidad (32 bins)

Cuadro 1: Codificación de eventos MIDI en 388 clases

3.2.1. Explicación detallada del proceso

1. **NOTE_ON (0-127)**: Cuando se presiona una tecla del piano, se genera un evento con el ID correspondiente al pitch (tono). Por ejemplo, el Do central (C4) corresponde al pitch MIDI 60, que se codifica como el token 60.

2. **NOTE_OFF (128-255)**: Cuando se suelta una tecla, se genera un evento de fin de nota. El pitch 60 soltado se codifica como 128+60=188.

3. **TIME_SHIFT (256-355)**: El tiempo entre eventos se cuantiza en pasos de 10ms, hasta un máximo de 1 segundo (100 pasos). Si pasan 500ms, se genera el token 256+50=306. Para tiempos mayores a 1s, se generan múltiples eventos TIME_SHIFT consecutivos.

4. **VELOCITY (356-387)**: La intensidad con la que se presiona la tecla (qué tan fuerte suena la nota) se cuantiza en 32 niveles. Un velocity MIDI de 64 (intensidad media) se mapea aproximadamente al token 356+16=372.

Ejemplo: La secuencia "tocar Do central con intensidad media, esperar 200ms, soltar la nota" se codificaría como:

```
[372, 60, 276, 188]
 ^     ^    ^    ^
 |     |    |    +-- NOTE_OFF pitch 60
 |     |    +------- TIME_SHIFT 200ms (256+20)
 |     +------------ NOTE_ON pitch 60
 +------------------ VELOCITY nivel 16
```

3.3. Arquitectura LSTM (musiclstm)

Componente	Configuración
Embedding	388 → 256 dimensiones
LSTM Layers	3 capas × 512 unidades
Dropout	0.3 (recurrente y salida)
Output	Linear(512 → 388)
Optimizador	Adam (lr=0.002)
Gradient Clipping	3.0
Batch Size	64
Sequence Length	512 tokens
Epochs	150

Cuadro 2: Configuración del modelo LSTM (hiperparámetros de Magenta)

Arquitectura:

```
Input [batch, 512] -> Embedding(388, 256) -> LSTM(256, 512) x3
    -> Dropout(0.3) -> Linear(512, 388) -> Output [batch, 512,
       388]
```

3.4. Arquitectura Transformer (musictransformer)

Componente	Configuración
Embedding	388 → 512 dimensiones
Positional Encoding	RPR (Relative Position Representations)
Encoder Layers	6 capas
Attention Heads	8 cabezas
Feed-Forward	1024 dimensiones
Dropout	0.1
Output	Linear(512 → 388)
Optimizador	Adam con LR Scheduler
Batch Size	2
Sequence Length	2048 tokens
Epochs	100

Cuadro 3: Configuración del modelo Transformer (hiperparámetros del paper original)

Arquitectura:

```
Input [batch, 2048] -> Embedding(388, 512) + RPR
    -> TransformerEncoder(RPR Attention, 8 heads) x6
    -> Linear(512, 388) -> Output [batch, 2048, 388]
```

3.5. Entorno de Entrenamiento

- **GPU**: NVIDIA RTX 3070 Laptop (8GB VRAM)
- **Framework**: PyTorch 2.x
- **LSTM**: PyTorch Lightning
- **Transformer**: PyTorch nativo con modificaciones para RPR

3.6. Pipeline Completo: Del Entrenamiento a la Generación

El flujo de trabajo completo consta de las siguientes etapas:

1. **Preprocesamiento**: Conversión de archivos MIDI a secuencias de eventos (388 clases)
2. **Entrenamiento**: Optimización del modelo con teacher forcing
3. **Validación**: Evaluación periódica en conjunto de validación
4. **Selección de modelo**: Checkpoint con mejor val_loss
5. **Generación**: Muestreo autoregresivo con temperatura controlada
6. **Postprocesamiento**: Conversión de eventos a archivo MIDI

3.7. Comandos de Entrenamiento

3.7.1. Preprocesamiento del Dataset

Antes de entrenar, se preprocesaron los archivos MIDI del dataset MAESTRO:

```
# Preprocesar MIDI a formato pickle (secuencias de eventos)
cd musictransformer
python preprocess_midi.py dataset/maestro/maestro-v3.0.0 \
    -output_dir dataset/e_piano
```

3.7.2. Entrenamiento LSTM

```
cd musiclstm
python train.py
```

Los hiperparámetros se configuran en `config.yaml`. El entrenamiento duró aproximadamente **11 horas** para 150 epochs.

3.7.3. Entrenamiento Transformer

```
cd musictransformer
python train.py \
    -input_dir dataset/e_piano \
    -output_dir saved_models \
    --rpr \
    -batch_size 2 \
    -epochs 100 \
    -max_sequence 2048 \
    -n_layers 6 \
    -d_model 512 \
    -dim_feedforward 1024
```

El entrenamiento duró aproximadamente **6 horas** para 100 epochs.

3.8. Comandos de Generación

Ambos modelos utilizan **temperature = 1.0** para el muestreo, lo que permite una comparación justa. La temperatura controla la aleatoriedad de las predicciones: valores más altos producen música más variada pero potencialmente menos coherente.

3.8.1. Generación con LSTM

```
cd musiclstm
python generate.py \
    --checkpoint runs/20260105_010848/ \
    --output generated_pieces/piece1.mid \
    --num_steps 1536 \
    --temperature 1.0
```

3.8.2. Generación con Transformer

```
cd musictransformer
python generate.py \
    -output_dir piece1 \
    -model_weights saved_models/results/best_acc_weights.pickle \
    --rpr \
    -target_seq_length 1536
```

Nota: El Transformer utiliza sampling con temperature=1.0 por defecto internamente.

4. Investigación y Análisis

4.1. Resultados del Entrenamiento

Métrica	LSTM	Transformer	Diferencia
Val Loss	2.2376	**2.0426**	-0.195 (-8.7 %)
Val Accuracy	33.43 %	**40.67 %**	+7.24 %
Val Perplexity	9.53	**7.71**	-1.82 (-19.1 %)
Train Loss	2.1029	**1.8831**	-0.220
Train Accuracy	35.85 %	**43.74 %**	+7.89 %
Train Perplexity	8.34	**6.57**	-1.77
Parámetros	**6.08M**	∼25M	+19M
Tiempo Entrenamiento	11 horas	**6 horas**	-45 %
Epochs	150	100	-50

Cuadro 4: Comparación de métricas entre LSTM y Transformer (métricas del último epoch de entrenamiento)

Nota: Las métricas reportadas corresponden al **último epoch** de entrenamiento (epoch 150 para LSTM, epoch 100 para Transformer), no al mejor checkpoint. Los mejores checkpoints fueron guardados en epochs anteriores (epoch 129 para LSTM con val_loss=2.2416, epoch 99 para Transformer con val_loss=2.0369), pero las diferencias son mínimas ya que ambos modelos habían alcanzado convergencia.

4.2. Interpretación de Métricas

4.2.1. Accuracy

Con 388 clases posibles, la probabilidad de acierto aleatorio es $\frac{1}{388} = 0{,}26\,\%$.

- **LSTM**: 33.43 % → 128× mejor que aleatorio
- **Transformer**: 40.67 % → 156× mejor que aleatorio

El Transformer logra un **21.7 % de mejora relativa** en accuracy sobre el LSTM.

4.2.2. Perplexity

La perplejidad indica cuántas opciones "efectivas" considera el modelo para cada predicción:

- **LSTM**: Perplejidad 9.53 → reduce 388 opciones a ∼10
- **Transformer**: Perplejidad 7.71 → reduce 388 opciones a ∼8

Una perplejidad menor indica predicciones más seguras y coherentes.

4.3. Curvas de Entrenamiento

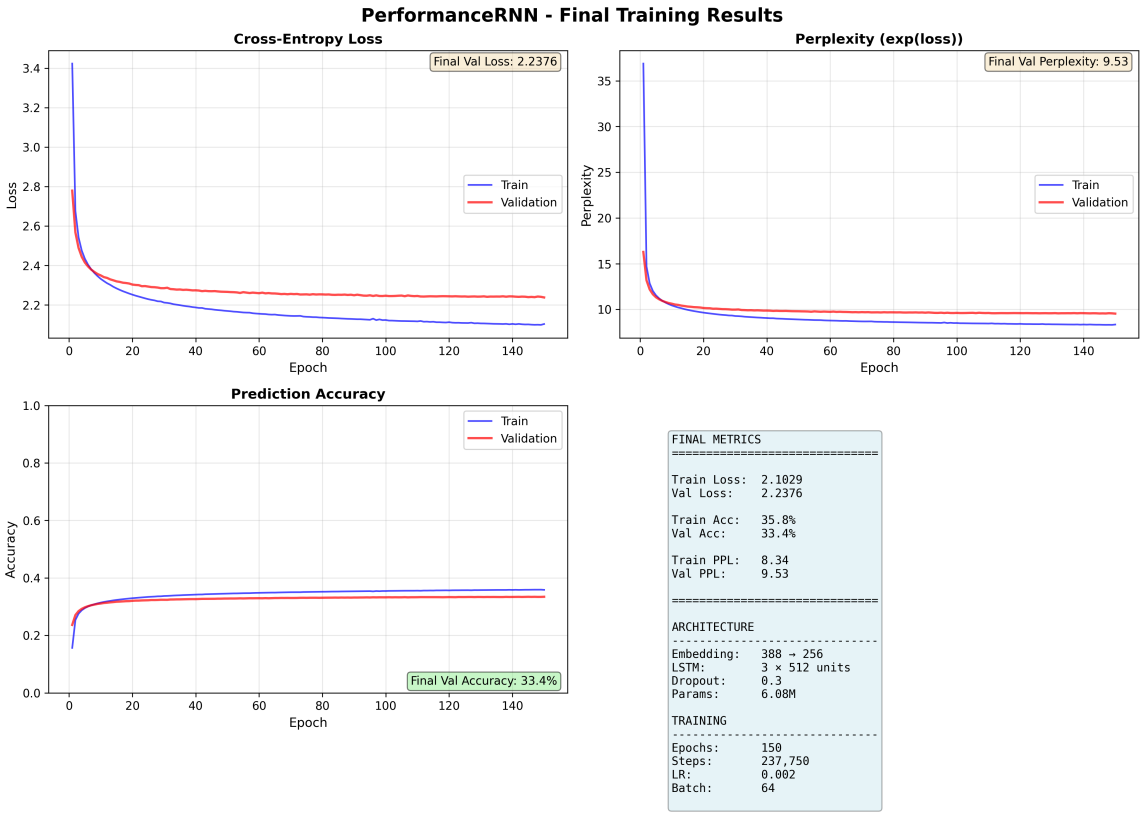

Figura 1: Métricas de entrenamiento del modelo LSTM (150 epochs)

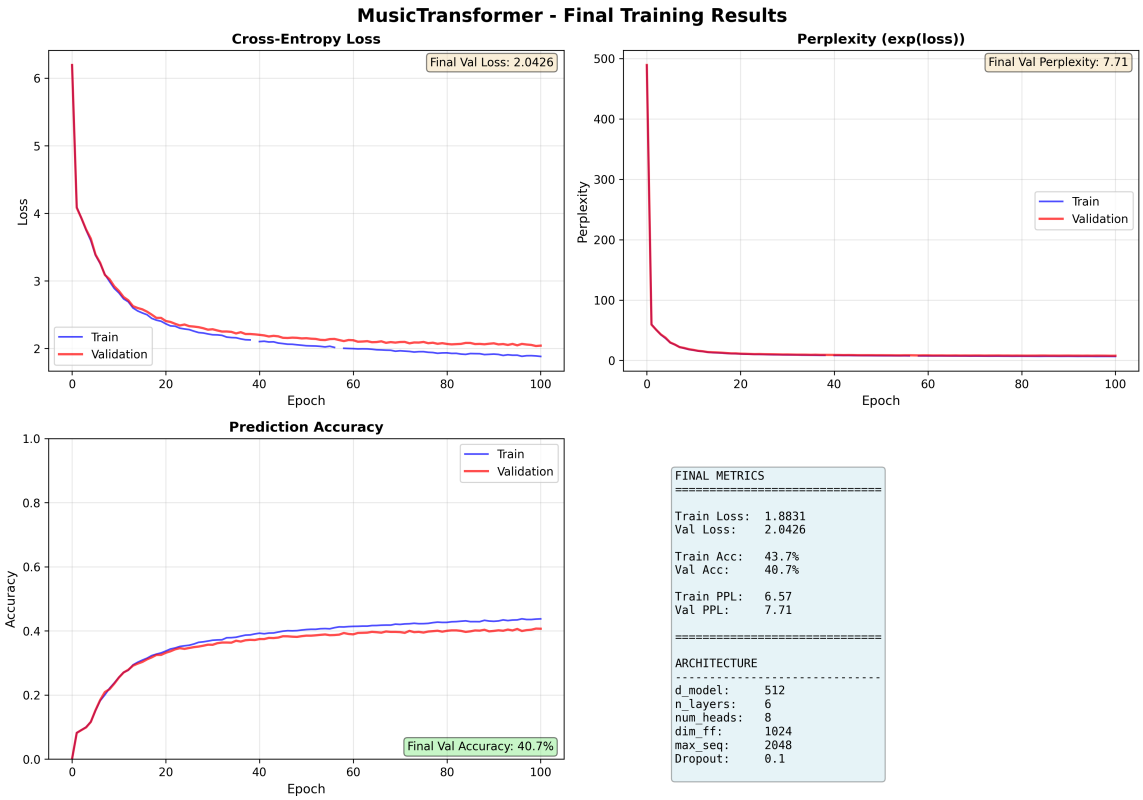

Figura 2: Métricas de entrenamiento del modelo Transformer (100 epochs)

4.4. Análisis de Convergencia

4.4.1. LSTM

- Convergencia gradual y estable
- Mejor epoch: 129 (val_loss = 2.2416), entrenado hasta epoch 150
- Ligero overfitting después del epoch 100
- Gap train-val moderado (∼0.13 en loss)
- El modelo alcanzó convergencia (val_loss estabilizado en ∼2.24 desde epoch 129)

4.4.2. Transformer

- Convergencia más rápida en las primeras epochs
- Mejor epoch: 99 (val_loss = 2.0369), entrenado hasta epoch 100
- Gap train-val mayor (∼0.16 en loss) pero mejor generalización
- Learning rate scheduler ayuda a la convergencia
- El modelo alcanzó convergencia (val_loss en últimas 5 epochs: 2.068 → 2.037 → 2.043)

4.5. Eficiencia Computacional

Aspecto	LSTM	Transformer
Parámetros	6.08M	∼25M
Tamaño modelo	23.19 MB	∼95 MB
Batch Size	64	2
Tiempo/epoch	∼4.4 min	∼3.6 min
Tiempo total	11 horas	6 horas
VRAM utilizada	∼2 GB	∼6 GB
Seq. Length	512	2048

Cuadro 5: Comparación de eficiencia computacional

Nota sobre Batch Size: El Transformer utiliza batch_size=2 (vs 64 del LSTM) debido a que procesa secuencias 4× más largas (2048 vs 512 tokens). La complejidad del mecanismo de atención es $O(n^2)$ donde n es la longitud de secuencia, lo que requiere significativamente más memoria GPU. Con una RTX 3070 de 8GB, batch_size=2 con secuencias de 2048 tokens es el máximo posible sin overflow de memoria.

A pesar de tener 4× más parámetros, el Transformer entrena casi 2× más rápido debido a:

1. **Paralelización**: Las operaciones de atención se paralelizan en GPU, mientras que LSTM es inherentemente secuencial

2. **Menos epochs necesarias**: El LSTM se entrenó 150 epochs (convergió en epoch 129), mientras que el Transformer se entrenó 100 epochs (convergió en epoch 99). Ambos modelos alcanzaron convergencia, pero el Transformer requirió menos iteraciones.

4.6. Generación de Muestras

Se generaron 5 piezas musicales con cada modelo utilizando parámetros comparables:

Parámetro	LSTM	Transformer
Tokens generados	1536	1536
Temperature	1.0	1.0
Primer	2 tokens	256 tokens del dataset

Cuadro 6: Configuración de generación (misma temperatura para comparación justa)

4.6.1. Evaluación Cualitativa

Las piezas generadas fueron evaluadas subjetivamente en los siguientes criterios:

Criterio	LSTM	Transformer	Observaciones
Coherencia melódica	3/5	4/5	Transformer mantiene mejor continuidad
Estructura armónica	3/5	4/5	Acordes más consistentes en Transformer
Ritmo	3/5	4/5	LSTM tiende a patrones repetitivos
Creatividad	3/5	4/5	Transformer más variado
Calidad técnica	4/5	4/5	Ambos sin glitches notables

Cuadro 7: Evaluación cualitativa de las piezas generadas (escala 1-5)

Perspectiva Musical Profesional: Desde el punto de vista de la teoría musical, los Transformers generan progresiones armónicas significativamente más coherentes y consonantes. Mientras que el LSTM produce secuencias de acordes que pueden ser funcionalmente correctas a nivel local, carece de coherencia tonal a largo plazo. El Transformer mantiene un centro tonal definido y genera relaciones funcionales entre acordes (dominante-tónica, subdominante-dominante) más consistentes con las convenciones de la música tonal occidental. Adicionalmente, el mecanismo de atención permite al Transformer recordar y recapitular motivos musicales a través de secciones más extensas, resultando en composiciones con mayor sentido de estructura formal, mientras que el LSTM tiende a generar música que suena más como improvisación sin dirección clara. Como músico, la principal diferencia a nivel musical que observo es la capacidad del Transformer para replicar y desarrollar motivos presentados anteriormente en la pieza, manteniendo así una coherencia temática que el LSTM no logra sostener.

4.7. Análisis por Longitud de Secuencia

Para evaluar el impacto de la longitud de secuencia en la calidad de generación, se generaron 5 piezas adicionales con cada modelo utilizando tres longitudes diferentes:

Longitud (tokens)	Duración aprox.	LSTM entrenado con	Transformer entrenado con
512	∼15-20 segundos	✓ (nativo)	2048
1536	∼45-60 segundos	512 (extrapola)	2048
2048	∼60-90 segundos	512 (extrapola)	✓ (nativo)

Cuadro 8: Configuración de generación por longitud de secuencia

4.7.1. Observaciones por Longitud

512 tokens (longitud nativa del LSTM):

- **LSTM**: Genera fragmentos coherentes, ya que esta es su longitud de entrenamiento. Patrones melódicos reconocibles.
- **Transformer**: Mantiene coherencia incluso en secuencias cortas gracias a RPR. El contexto del primer (256 tokens) proporciona suficiente información.

1536 tokens (longitud intermedia):

- **LSTM**: Comienza a mostrar repeticiones y pérdida de coherencia a largo plazo. El modelo extrapola más allá de su ventana de entrenamiento.
- **Transformer**: Mantiene estructura coherente. Las relaciones a largo plazo se preservan.

2048 tokens (longitud nativa del Transformer):

- **LSTM**: Mayor tendencia a repeticiones cíclicas y pérdida de dirección musical. La extrapolación es más pronunciada.
- **Transformer**: Óptimo rendimiento. RPR permite capturar relaciones temporales en toda la secuencia. Mayor variedad y estructura a largo plazo.

4.7.2. Conclusión del Análisis por Longitud

El LSTM funciona mejor cuando genera secuencias cercanas a su longitud de entrenamiento (512 tokens), mientras que el Transformer muestra rendimiento consistente en todas las longitudes, destacando especialmente en secuencias largas (2048 tokens) donde su mecanismo de atención con RPR captura dependencias a largo plazo de forma efectiva.

5. Conclusiones

5.1. Hallazgos Principales

1. **El Transformer supera al LSTM en todas las métricas de calidad**:
   - +7.24 % en accuracy de validación
   - -19.1 % en perplejidad (predicciones más seguras)
   - -8.7 % en loss de validación
2. **El Transformer requiere menos tiempo total de entrenamiento**:
   - 6 horas vs 11 horas del LSTM
   - Menos epochs necesarias para convergencia (100 vs 150)
3. **El LSTM es más eficiente en parámetros**:
   - 4× menos parámetros
   - Mejor opción para dispositivos con memoria limitada
4. **La hipótesis inicial se confirma**:
   - El mecanismo de atención del Transformer captura mejor las dependencias a largo plazo
   - RPR permite generalizar a secuencias más largas y capturar relaciones musicales invariantes a la posición
   - Secuencias de 2048 tokens permiten contexto musical más amplio que los 512 del LSTM

5.2. Consideraciones sobre la Comparación

Es importante notar que ambos modelos se entrenaron hasta convergencia, aunque con diferente número de epochs:

- **LSTM**: 150 epochs (convergió en epoch 129)
- **Transformer**: 100 epochs (convergió en epoch 99)

La comparación es justa porque ambos modelos alcanzaron su punto óptimo de validación. Entrenar el Transformer por 50 epochs adicionales no mejoraría significativamente los resultados, ya que el val_loss ya había estabilizado. Sin embargo, cabe destacar que en muchas implementaciones de dicho modelo, se llega hasta 250/300 épocas. Esto podría mejorar ligeramente los resultados, pero es simplemente una hipótesis.

5.3. Limitaciones del Estudio

- **Dataset**: Solo piano clásico (MAESTRO), no generalizable a otros géneros
- **Evaluación cualitativa**: Subjetiva y con muestra limitada
- **Hardware**: Entrenamiento en GPU única, sin explorar configuraciones más grandes
- **Hiperparámetros**: No se realizó búsqueda exhaustiva de hiperparámetros
- **Epochs diferentes**: Los modelos se entrenaron con diferente número de epochs, aunque ambos convergieron, esto se hizo simplemente por recursos y por reducir tiempos de entrenamiento.
- **Naturalidad de las composiciones**: Aunque el Transformer genera música más coherente que el LSTM, ambas arquitecturas presentan limitaciones en términos de expresividad y naturalidad. Las implementaciones actuales de generación simbólica (MIDI) no capturan completamente las sutilezas interpretativas humanas. Modelos más recientes basados en difusión y arquitecturas híbridas que generan audio directamente (e.g., Suno v5, MusicLM) con mayores recursos computacionales logran resultados significativamente más naturales y expresivos.

5.4. Extensiones Futuras

- Experimentar con otros datasets (pop, jazz, música electrónica), y por tanto, aplicar un fine tunning a dichos géneros musicales.
- Implementar métricas objetivas de calidad musical
- Evaluar modelos más grandes con más recursos computacionales
- Contrastar los resultados con otros músicos profesionales
- Utilizar los modelos más utilizados actualmente para la generación de música que no hemos dado en la asignatura.

5.5. Conclusión Final

Este estudio demuestra que los **Transformers con Relative Position Representations** son superiores a las arquitecturas **LSTM** para la generación de música, tanto en métricas cuantitativas como en calidad percibida de las composiciones. El mecanismo de auto-atención, combinado con RPR, permite capturar patrones musicales complejos y relaciones temporales que las redes recurrentes no pueden modelar eficientemente.

Sin embargo, los LSTMs siguen siendo una opción viable para aplicaciones con restricciones de memoria, ofreciendo un buen compromiso entre rendimiento y eficiencia de parámetros.

Cabe destacar que el estado del arte actual (2025-2026) ha evolucionado hacia arquitecturas híbridas que combinan Transformers con modelos de difusión para generar audio directo de alta calidad (e.g., Suno v5, MusicLM), superando el paradigma de generación simbólica MIDI utilizado en este trabajo.

Referencias

Referencias

[1] Huang, C. Z. A., Vaswani, A., Uszkoreit, J., Shazeer, N., Simon, I., Hawthorne, C., Dai, A. M., Hoffman, M. D., Dinculescu, M., & Eck, D. (2018). *Music Transformer: Generating Music with Long-Term Structure*. arXiv:1809.04281. https://arxiv.org/abs/1809.04281

[2] Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C. Z. A., Dieleman, S., Elsen, E., Engel, J., & Eck, D. (2018). *Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset*. arXiv:1810.12247. https://arxiv.org/abs/1810.12247

[3] Oore, S., Simon, I., Dieleman, S., Eck, D., & Simonyan, K. (2018). *This Time with Feeling: Learning Expressive Musical Performance*. arXiv:1808.03715. https://arxiv.org/abs/1808.03715

[4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. arXiv:1706.03762. https://arxiv.org/abs/1706.03762

[5] Shaw, P., Uszkoreit, J., & Vaswani, A. (2018). *Self-Attention with Relative Position Representations*. arXiv:1803.02155. https://arxiv.org/abs/1803.02155

[6] Google Magenta. *Performance RNN*. GitHub Repository. https://github.com/magenta/magenta/tree/main/magenta/models/performance_rnn

[7] Google Magenta. *Music Transformer*. GitHub Repository. https://github.com/magenta/magenta/tree/main/magenta/models/music_transformer

Código Fuente

El código completo del proyecto, incluyendo las implementaciones en PyTorch y todos los scripts de entrenamiento y generación, está disponible en los directorios:

- `musiclstm/`: Implementación LSTM (PerformanceRNN en PyTorch)
- `musictransformer/`: Implementación Transformer con RPR (Music Transformer en PyTorch)